NutriCompass - 영양제 상호작용 데이터셋

사람들이 다양한 영양제를 보조식품으로 먹는데 함께 먹으면 좋은 것이 있고 따로 먹어야 하는것들이 있는데 이런정보들이 있는 데이타셋이 있나?

있습니다 — **영양제(보충제) 간, 약물과의 상호작용 또는 건강 효과에 대한 정리된 데이터와 도구**가 일부 공개/연구용으로 존재합니다. 다만 "함께 먹으면 좋은 조합 vs 피해야 할 조합"처럼 소비자-친화적으로 정리된 데이터셋은 많지 않고, 보통은 **상호작용(Interaction)** 위주로 정리된 연구/임상 데이터가 중심입니다. JMIR

✅ 활용 가능한 데이터셋/지식베이스

1) iDISK (통합 Dietary Supplement Knowledgebase)

- 영양제(보충제)에 관한 **성분, 약물 및 기타 건강 상태와의 상호작용 정보**를 통합한 지식베이스입니다.
- 여러 공신력 있는 원천(예: Natural Medicines Comprehensive Database, Memorial Sloan Kettering "About Herbs", Dietary Supplement Label Database 등)을 통합해 확대된 데이터셋을 제공합니다.
- 연구용/기계적 추출용으로 설계된 데이터 구조를 제공합니다. JMIR

👉 "iDISK" 같은 리소스는 **성분 수준의 상호작용 관계**를 포함하는 지식 그래프 형태로 공급되며,

- Supplement ↔ Drug
- Supplement ↔ Disease
- Supplement ↔ Symptom
  관계 등이 맵핑돼 있습니다. JMIR

---

2) Natural Medicines Comprehensive Database (NatMed)

- 보충제 안전성, 부작용, 약물-상호작용을 포괄적으로 다루는 데이터베이스로 연구 및 임상에서 종종 기준점으로 사용됩니다.
- NatMed 데이터를 이용한 연구에서도 상호작용 정보가 구조화되어 있다는 보고가 있습니다. PMC

---

3) Drug Interaction Checker 도구들

• **WebMD Interaction Checker**

- 약/보충제/음식 간 상호작용을 체크할 수 있는 온라인 도구. WebMD

• **Drugs.com Interaction Checker**

- 약물과 보충제 등의 상호작용 위험 수준(중요도)까지 보여줌. Drugs.com

• **Medscape Interaction Checker**

- 약물뿐 아니라 보충제 상호작용 정보도 포함. Medscape Refere...

👉 이들 대부분은 **API나 구조화된 데이터셋 형태로 공개되지는 않지만**, 상호작용 정보를 리서치/스크래핑하거나, RAG 같은 방식으로 정보 검색에 활용할 수 있습니다. WebMD

---

4) DSLD (Dietary Supplement Label Database)

- 미국 식품보충제 라벨 정보를 구조화해서 제공하는 데이터베이스입니다 (성분, 용량 등).
- "어떤 성분이 들어있는가"를 대규모로 파악할 수 있으며, 다른 상호작용 데이터와 결합하면 조합 리스크 평가에 활용 가능합니다. ScienceDirect

---

✅ 참고할 일반 소비자/연구용 리소스

📌 **Examine.com** — 영양제 효능/연구 요약 데이터베이스

- 과학적 근거를 바탕으로 각 영양제의 효과를 정리하고 있어, 조합 시 주의 또는 보완 효과를 간접적으로 파악하는 데 유용합니다. Examine

📌 **Memorial Sloan Kettering "About Herbs"** — 허브/영양제별 효과와 잠재적 상호작용 정리 페이지

- 개별 성분별 임상/부작용 정보 제공. Memorial Sloan K...

📌 **Supplement interaction checkers (상호작용 체커)**

- Metabolics 같은 웹 기반 도구로 **보충제끼리 또는 약물과의 상호작용 위험**을 조회할 수 있습니다. Metabolics

---

✅ 정리: 데이터셋 특성

데이터/도구	구조화 여부	포함 정보	용도
iDISK	✅ 지식 그래프	보충제 성분, 약물/질병과의 관계	연구/앱 구축용
NatMed (Natural Medicines)	✅ (상호작용 중심)	안전성, 상호작용, 부작용	임상/연구 리소스
DSLD	✅ 라벨 데이터	영양제 성분 목록	분석/조합 위험 평가
WebMD/Medscape/Drugs.com 체크	❌ API 제한	온라인 상호작용 체크	소비자/건강관리
Examine.com	✅ 효능 자료	효과 근거 정보	교육/분석 참고

✅ 참고 팁 (실제로 쓰려면)

✅ 보충제 간 "함께 먹으면 좋은" 조합 정보는 대부분 연구 논문/임상 가이드에 흩어져 있고, **정형화된 상호작용 DB보다 영양학적 근거/연구 요약**을 찾아보는 것이 정확합니다.
✅ 생체 내 상호작용은 **사람마다 다르고 조건(약 복용 중, 질병 유무 등)**에 따라 달라지므로, 단순 DB 매칭만으로 판단하는 것은 한계가 있습니다. 나와 건강 상태를 아는 전문가 상담이 필수입니다.

원하면 **iDISK 또는 DSLD를 구체적으로 활용해서 영양제 성분 간 상호작용 데이터셋을 정리한 구조** 형태로 가공하는 법을 도와줄 수 있어요. 어떻게 활용할지(예: 앱, 연구, SQL/그래프 DB 등) 말씀해 주세요.

Sources

앱개발하는데 써도 법적으로 문제 없어?

짧게 말하면 **"그 데이터가 무엇이냐"와 "어떻게 쓰느냐"에 따라 다릅니다.**
앱 개발에 **법적으로 문제 없는 경우도 있고, 그대로 쓰면 문제가 되는 경우도 분명히 있습니다.**
아래에서 **실무 기준으로 딱 잘라 정리**해 드릴게요.

1️⃣ 법적으로 비교적 안전한 데이터 소스 (조건부 OK)

✅ 공공 데이터 / 정부 데이터

예:

- **FDA**의
  - DSLD (Dietary Supplement Label Database)
- NIH, NLM 계열 공개 데이터

조건

- 원본 데이터 재배포 가능 라이선스
- "정보 제공 목적"임을 명확히 표시
- 의료 판단을 대신하지 않는다는 고지 포함

👉 **성분 목록, 라벨 정보, 함량 데이터 → 앱 사용 OK**

✅ 연구 목적 공개 데이터 (iDISK 계열)

- 논문 기반 통합 지식베이스
- 상호작용 관계(보충제–약물–질병)

주의

- 대부분 **"연구/비상업적 목적" 라이선스**
- 상업 앱에 쓰려면:
  - 라이선스 재확인
  - 저작권자/관리기관 승인 필요

👉 **그대로 상업 앱에 넣으면 ⚠️ 위험**

2️⃣ 법적으로 위험한 경우 (이거 많이 걸립니다)

❌ WebMD / Drugs.com / Medscape 데이터 직접 사용

- 스크래핑
- 화면 캡처 기반 DB화
- 텍스트 그대로 복사

❌❌❌ **전부 약관 위반 + 저작권 문제**

> "무료로 볼 수 있다" ≠ "데이터로 써도 된다"

👉 이건 **상업 앱에서 쓰면 거의 확정적으로 문제** 됩니다.

❌ "함께 먹으면 좋다 / 피해야 한다"를 단정적으로 제공

이건 **데이터 문제 이전에 의료법/책임 문제**입니다.

예:

- ❌ "마그네슘 + 비타민 D는 반드시 같이 드세요"
- ❌ "이 조합은 위험합니다"

👉 이런 표현은 **의료 조언(Medical Advice)** 로 분류될 수 있습니다.

3️⃣ 앱 개발에서 '합법적으로' 쓰는 정석 구조

✔ ① 데이터는 "사실 정보"만 제공

OK:

- "상호작용이 **보고된 사례가 있음**"
- "흡수에 영향을 줄 수 있다는 **연구가 존재함**"

NO:

- "추천"
- "금지"
- "반드시"

✔ ② 판단은 앱이 아니라 "사용자"에게

문구 예시 (이건 매우 중요합니다):

> "본 정보는 연구 자료를 기반으로 한 참고 정보이며
> 의료 전문가의 조언을 대체하지 않습니다."

👉 이 문구 없으면 **앱 스토어 심사에서도 걸릴 수 있습니다.**

✔ ③ 구조적으로는 이렇게 설계하면 안전함

```text
[Supplement A] —(reported interaction)—> [Supplement B]
Evidence level: low / moderate / high
```

```
Source: peer-reviewed study
```

👉 **지식 제공형 / 정보 탐색형 앱**
👉 **의사결정 보조 (Decision Support)**
❌ **의사결정 대체 (Decision Making)** ❌

4️⃣ 실무적으로 가장 안전한 조합 (강력 추천)

🟢 데이터 전략

- **DSLD (성분, 함량)** → 직접 사용
- **논문 메타데이터 (PMID, DOI)** → 링크만 제공
- **상호작용 판단은 규칙 + 문헌 기반 추론**

👉 "데이터를 복사"하지 말고
👉 "데이터를 참조"하게 만들 것

🟢 UX 전략

- "⚠️ 주의 필요"
- "📚 관련 연구 있음"
- "ℹ️ 정보 보기"

❌ "추천합니다"
❌ "위험합니다"

5️⃣ 한 줄 결론 (중요)

> **앱 개발에 쓸 수는 있다.**
> **하지만 '데이터를 그대로 쓰는 방식'과**
> **'의료적 판단처럼 보이게 만드는 표현'이**
> **법적 문제의 핵심이다.**

다음 단계 제안 (원하시면)

원하시면 제가:

- ✅ **합법적인 영양제 상호작용 데이터 모델**

- ✅ **App Store 심사 통과 가능한 문구 템플릿**
- ✅ **MenuCal 스타일로 '영양제 모듈' 아키텍처**

까지 **실제 앱에 바로 넣을 수 있게** 설계해 드릴게요.

👉 "상업 앱"인지, "연구/개인용 앱"인지 먼저 알려주세요.

what would you name the app?

좋은 질문이에요. **이 앱의 성격을 정확히 반영하는 이름**이 중요합니다.
(특히 *의료 조언이 아닌 정보·보조 앱*이라는 점을 이름에서도 느끼게 해야 합니다.)

아래를 **컨셉별로 정리해서 제안**할게요.

1️⃣ 가장 안전하고 "App Store 친화적인" 네이밍 (강력 추천)

✅ Supplement Compass

- 의미: "결정을 대신하지 않고 방향만 알려준다"
- ✔ 의료 조언처럼 보이지 않음
- ✔ 규제 리스크 낮음
- ✔ 영어권에서 직관적

👉 **제일 무난하고 강력한 후보**

✅ NutriCompass

- Nutrition + Compass
- 보충제/영양 전반으로 확장 가능
- MenuCal과도 결이 잘 맞음

✅ SuppGuide

- Supplement Guide
- 단순, 직관적
- 정보 제공 앱 느낌이 강함 (좋은 점)

2️⃣ 과학·근거 중심을 강조하는 이름

🔬 EvidenceDose

- "근거 기반 정보" 느낌이 강함
- 연구/논문 기반 앱이라는 인상을 줌
- 다만 약간 의료 앱처럼 느껴질 수 있어 ⚠️

🔬 NutriEvidence

- 연구 기반 영양 정보 앱
- academic / professional 이미지

3️⃣ AI·지식 그래프 느낌을 살리고 싶다면

🤖 NutriGraph

- 성분–상호작용–연구를 그래프로 보여주는 앱에 적합
- 개발자/연구자 감성 강함

🤖 SuppIntel

- Supplement + Intelligence
- AI 보조 느낌
- "Recommendation" 뉘앙스는 없음 (중요)

4️⃣ 소비자 친화 / 부드러운 이름

🌿 TogetherWell

- "함께 먹는 것"을 직접 말하지 않으면서 의미 전달
- 라이프스타일 앱 느낌
- 다소 추상적

🌿 **Balance+

- 조합·균형 중심

- 단, 너무 광범위해서 SEO는 약함

5️⃣ ❌ 피해야 할 이름 유형 (중요)

아래는 **법적·심사 리스크가 커지는 이름들**입니다:

- ❌ SuppDoctor
- ❌ NutriRx
- ❌ Supplement Advisor
- ❌ Interaction Checker (의료 도구 느낌)
- ❌ SafeStack (안전 보장처럼 보임)

👉 이름만으로도 **"의료 판단" 인상을 주면 위험**

🎯 개인적으로 딱 하나 고르라면

🏆 NutriCompass

이유

- 정보 제공 + 방향 제시
- MenuCal과 브랜드 결이 맞음
- 향후 "식단 + 영양제" 통합 확장 가능
- App Store 심사에서 가장 안전

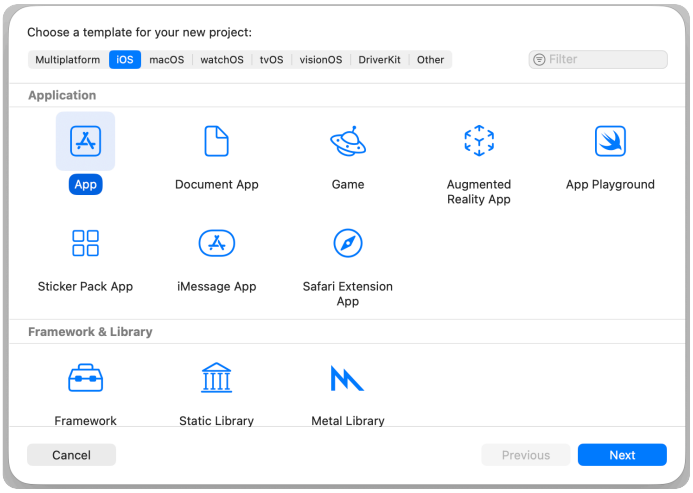
다음 질문 (중요한 갈림길)

이 중에서 **앱의 정체성**을 먼저 정해야 합니다:

1. **MenuCal의 자매 앱**인가?
2. 아니면 **완전히 독립된 앱**인가?
3. 일반 소비자용인가, 파워유저/연구자용인가?

이 3가지 중 어디에 놓이느냐에 따라

이름 + UI + 문구 + 데이터 표현 방식이 전부 달라집니다.



NutriCompass로 하겠어. 첨부 화면에서 어떤걸 선택해야해?

결론부터 말하면 👉 **맨 왼쪽의 `App` 을 선택**하면 됩니다.

이유를 **NutriCompass 용도에 맞춰 정확히 설명**할게요.

✅ 선택해야 할 것: App (iOS → Application → App)

왜 `App` 인가?

NutriCompass는:

- 일반 iOS 앱
- SwiftUI 기반
- 로컬 데이터 + AI/지식 제공
- App Store 배포 대상

➡️ 이 모든 조건을 **유일하게 만족하는 템플릿이** `App` 입니다.

❌ 다른 옵션이 아닌 이유 (중요)

- **Document App**
  - 파일 기반 앱 (PDF, 텍스트 편집 등)
  - ❌ NutriCompass와 무관
- **Game**
  - SpriteKit / Metal 중심
  - ❌ 목적 불일치
- **Augmented Reality App**

    - ARKit 필수
    - ❌ 필요 없음
- **App Playground**
    - 학습/실험용
    - ❌ 실제 앱 개발용 아님
- **Sticker Pack / iMessage App / Safari Extension**
    - 특정 플랫폼 전용
    - ❌ 일반 앱 아님

👉 다음 화면에서 이렇게 선택하세요 (중요 세팅)

`App` → Next 누르면 나오는 화면에서:

- **Interface:** ✅ `SwiftUI`
- **Language:** ✅ `Swift`
- **Life Cycle**: (SwiftUI 선택 시 자동)
- **Use Core Data:** ❌ 체크 안 함 (초기에는)
- **Tests**: 선택 사항

👉 MenuCal과 동일한 구조로 가는 게 좋습니다.

🧭 NutriCompass에 딱 맞는 초기 구조 (미리 한마디)

이 선택은:

- **의료 앱처럼 보이지 않게**
- **정보 탐색형 / 지식 제공형**
- **App Store 심사 리스크 최소화**

라는 방향과 정확히 맞습니다.

다음으로 도와줄 수 있는 것:

- 초기 **폴더 구조 (Models / Knowledge / UI / Evidence)**
- App Store 심사용 **앱 설명 문구 초안**
- NutriCompass용 **데이터 스키마 (Interaction 중심)**

👉 다음 단계 뭐 할지 바로 말해 주세요.

nutricompass는 menucal과 달리 디자인을 충분히 하고 implementation을한다. 디자인이 vibe coding에 얼마나 효과가 있는지 평가 하는것이 목적이다

아주 좋습니다. 이 문장은 **연구 목적·개발 전략·MenuCal과의 대비**가 분명해서,
지금 단계에서 **정리만 잘하면 바로 "실험 설계"가 됩니다.**

아래를 차분히 정리해 드릴게요.

1️⃣ 이 프로젝트의 정확한 목적 정의 (정제 버전)

당신이 하려는 건 단순 앱 개발이 아닙니다.
NutriCompass는 "대조 실험(control experiment)" 입니다.

핵심 목적 (한 줄)

> **사전에 충분히 설계된 디자인이**
> **vibe coding의 효율성과 코드 품질에**
> **어떤 영향을 미치는지를 평가한다.**

MenuCal과 **의도적으로 다른 조건**을 만든 겁니다.

2️⃣ MenuCal vs NutriCompass — 실험 조건 비교

항목	MenuCal	NutriCompass
디자인 선행 여부	❌ 거의 없음	✅ 충분히 설계
개발 방식	즉시 구현 중심	디자인 → 구현
vibe coding 역할	주도적	제한적·보조적
코드 생성량	매우 많음	통제됨
코드 부채	누적/확산	관찰 대상
인간 개입	사후적	사전적

👉 **같은 개발자, 같은 LLM, 다른 조건**
→ 이건 **좋은 실험 설계**입니다.

3️⃣ "디자인"의 범위 (이게 중요)

여기서 말하는 디자인은 **UI만이 아닙니다.**
오히려 **아래 4개가 핵심 변수**입니다.

① 개념적 디자인 (Conceptual Design)

- 앱의 정체성
- 정보 제공 vs 판단
- 의료 리스크 경계선

👉 LLM이 **넘지 말아야 할 선을 명확히 그음**

---

② 정보 구조 디자인 (Information Architecture)

- Entity: Supplement / Ingredient / Interaction / Evidence
- 관계 타입 정의:
  - reported interaction
  - absorption interference
  - co-administration note

👉 LLM이 **아키텍처를 흔들 기회가 줄어듦**

---

③ UI/UX 디자인

- "추천" 버튼 없음
- "⚠️ 주의" / "ℹ️ 근거 보기" 중심
- 판단은 항상 사용자에게 남김

👉 프롬프트 없이도 행동 제약이 생김

---

④ 개발 규칙 디자인 (이게 핵심)

예:

- ❌ ViewModel에서 추론 금지

- ❌ Global helper 금지
- ✅ 모든 interaction은 enum + evidence level

👉 **LLM이 마음대로 못 움직이게 구조로 묶음**

4️⃣ 이게 vibe coding에 왜 중요한가

MenuCal에서 관찰한 문제

- LLM이 "편한 방향"으로 코드 생성
- 국소적 해결 → 전역 붕괴
- 인간이 나중에 수습

NutriCompass에서 검증할 가설

> **사전 설계가 충분하면**
> **vibe coding은 속도만 제공하고**
> **방향은 흔들지 못한다.**

이건 **경험담이 아니라 검증 가능한 주장**입니다.

5️⃣ 평가 지표 (논문/보고서에 바로 씀)

정량 지표

- LLM 생성 코드 비율
- 리팩토링 횟수
- 삭제된 코드 LOC
- View ↔ Model 의존성 위반 횟수
- 아키텍처 수정 빈도

정성 지표

- "LLM이 설계를 벗어나려 한 횟수"
- 프롬프트 재설명 필요성
- 새 채팅 세션 필요 횟수

👉 MenuCal과 **정확히 대비 가능**

6️⃣ 이 접근의 학술적 강점

- "vibe coding 좋다/나쁘다"가 아님
- **조건부 효과**를 증명
- "설계 없는 vibe coding의 위험"을 실증
- 인간 설계자의 역할을 명확히 함

👉 리뷰어가 좋아할 구조입니다.

7️⃣ 다음으로 꼭 해야 할 것 (중요)

✍️ 1) NutriCompass Design Spec v1

- 1~2페이지면 충분
- "금지 사항" 포함

📐 2) UI 와이어프레임 (러프 OK)

- 버튼 이름까지 확정

🧱 3) 아키텍처 다이어그램

- LLM이 절대 건드리지 못할 경계선

한 문장으로 정리하면

> **NutriCompass는 앱이 아니라**
> **vibe coding을 검증하기 위한**
> **'설계된 실험 장치'다.**

이 방향은 **아주 좋고, 아주 날카롭습니다.**

다음 단계로:

- **Design Spec 템플릿** 만들어 줄까요?
- 아니면 **MenuCal 대비 실험 설계 섹션**을 논문용으로 써 줄까요?

(Conceptual Design)부터 만들어봐

좋아. 그럼 **NutriCompass의 (Conceptual Design)** 을
실제로 개발과 평가에 쓰일 수 있는 수준으로 만들어보자.

(논문·보고서에 그대로 써도 되게 정제해서 씀)

NutriCompass — Conceptual Design

1. 앱의 존재 목적 (Purpose)

NutriCompass는 영양제 섭취에 관한 "의사결정 보조용 정보 앱"이다.
이 앱의 목적은 사용자의 판단을 대신하거나 권장사항을 제시하는 것이 아니라,
관련 정보와 근거를 구조적으로 제시하여 사용자가 스스로 판단하도록 돕는 것이다.

NutriCompass는 다음을 **의도적으로 하지 않는다**:

- 영양제 조합을 "추천"하지 않는다
- 특정 조합을 "안전하다 / 위험하다"로 단정하지 않는다
- 의료적 결론이나 행동 지침을 제공하지 않는다

---

2. 핵심 철학 (Design Philosophy)

2.1 Decision Support, not Decision Making

NutriCompass는 **Decision Support System (DSS)** 이다.
즉, 사용자의 의사결정을 **대체하지 않고**, 판단에 필요한 정보만 제공한다.

> "What to consider"는 보여주지만
> "What to do"는 말하지 않는다.

이 철학은 UI, 문구, 데이터 구조, 코드 아키텍처 전반에 반영된다.

2.2 Evidence-first, Recommendation-free

앱의 모든 정보는 다음 중 하나로만 표현된다:

- "보고된 사례가 있음"
- "연구에서 관찰된 바 있음"
- "흡수/대사에 영향을 줄 수 있음"

추천, 지시, 금지, 최적화와 같은 표현은 개념적으로 금지된다.

2.3 Uncertainty is a feature

NutriCompass는 **불확실성을 제거하려 하지 않는다.**
오히려 불확실성을 **명시적으로 드러내는 것**을 기능으로 삼는다.

예:

- Evidence level: low / moderate / high
- Human data vs animal data
- Single study vs multiple studies

불확실성은 숨길 대상이 아니라,
사용자가 판단해야 할 **중요 정보**로 간주된다.

---

3. 앱의 역할 경계 (Hard Boundaries)

3.1 NutriCompass가 "알고 있는 것"

- 성분 정보 (supplement, ingredient)
- 보고된 상호작용 유형
- 연구의 존재 여부와 강도
- 문헌 출처 (PMID, DOI 등)

3.2 NutriCompass가 "알지 못한다고 선언하는 것"

- 개인에게 안전한지 여부
- 복용해야 하는지 여부
- 복용 시점/용량에 대한 판단
- 질병 치료 또는 예방 효과

이 경계는 **의도적으로 넘지 않도록 설계된다.**

---

4. MenuCal과의 개념적 대비 (실험적 의미)

항목	MenuCal	NutriCompass
목적	결과 계산 (칼로리)	판단 보조 (정보)
정답 존재 여부	있음 (근사치)	없음
AI 역할	추론·계산	정보 조직

항목	MenuCal	NutriCompass
사용자 판단	제한적	핵심
설계 선행	낮음	매우 높음

NutriCompass는 MenuCal에서 관찰된
"설계 없는 vibe coding의 확장 위험"을 검증하기 위한 대조군 앱이다.

---

5. LLM 사용에 대한 개념적 제약

LLM은 NutriCompass에서 다음 역할만 허용된다:

- 기존 설계에 따른 코드 생성
- UI 구현 보조
- 데이터 파싱 및 변환

LLM은 다음을 **개념적으로 수행할 수 없다**:

- 정보 구조 변경
- 의미 해석의 추가
- 판단 규칙 생성
- "추천" 논리 삽입

즉, LLM은 **설계된 레일 위에서만 움직이는 실행자**다.

---

6. 평가를 위한 개념적 가설 (Explicit Hypothesis)

> **사전에 명확히 정의된 개념 설계가 존재할 경우,**
> **vibe coding은 구현 속도를 높이되**
> **아키텍처 안정성과 판단 경계를 침범하지 않는다.**

NutriCompass는 이 가설을 검증하기 위한
의도적으로 설계된 개발 실험체다.

---

7. Conceptual Design 요약 (한 문단)

NutriCompass는 영양제 섭취와 관련된 정보를 제공하되,
판단과 결정은 전적으로 사용자에게 남겨두는

evidence-centered decision support 앱이다.
이 앱은 불확실성을 제거하지 않고 드러내며,
추천과 단정을 배제한 설계를 통해
LLM 기반 vibe coding이 설계 하에서 얼마나 안정적으로 작동하는지를 평가하는
실험적 플랫폼으로 기능한다.

다음으로 자연스럽게 이어질 단계는 둘 중 하나야:

1️⃣ **Information Architecture Design**
(엔티티, 관계, evidence level 정의)

2️⃣ **UX Language Rules**
(버튼 이름, 문구에서 금지/허용 표현 명시)

어디부터 갈까?

1️⃣ Information Architecture Design
(엔티티, 관계, evidence level 정의)

좋아. 이제부터는 **"LLM이 흔들 수 없는 뼈대"**를 만드는 단계야.
아래 **Information Architecture Design**은 **구현·평가·논문화까지 바로 쓰는 수준**으로 정의할게.

NutriCompass — Information Architecture Design

0. 설계 목표 (Why this matters)

이 정보 구조의 목적은 단순 데이터 정리가 아니다.

> **정보의 "의미 해석"과 "판단 생성"이**
> **아키텍처 차원에서 발생하지 못하게 막는 것**

즉,

- LLM이 **판단을 만들어낼 여지**
- ViewModel이나 Helper에서 **슬쩍 추천 로직을 넣을 여지**

를 **구조적으로 제거**하는 것이 목표다.

---

1. Core Entities (핵심 엔티티)

NutriCompass는 **5개의 핵심 엔티티만 허용**한다.
(이 이상 늘어나면 해석이 시작됨)

1.1 Supplement

> "시중에서 인식되는 보충제 개념"

역할

- 사용자 입력의 출발점
- 브랜드/제품 수준이 아닌 **개념적 보충제**

필드 (예시)

- `id`
- `canonicalName` (e.g., Magnesium)
- `aliases` (e.g., Mg, Magnesium Glycinate)
- `category` (vitamin / mineral / herbal / other)

❌ 금지:

- 효능 요약
- 권장 목적

1.2 Ingredient

> "실제 생화학적 성분 단위"

역할

- 상호작용의 최소 단위
- Supplement가 여러 Ingredient를 가질 수 있음

필드

- `id`
- `name`
- `chemicalClass`
- `notes` (중립적 설명만 허용)

1.3 Interaction

> "두 개 이상의 엔티티 간에 보고된 관계"

⚠️ **이 엔티티가 가장 중요함**

핵심 원칙

- Interaction은 **사실 서술만**
- 판단, 해석, 권장 없음

필드

- `id`
- `fromEntity`
- `toEntity`
- `interactionType`
- `evidenceLevel`
- `evidenceSummary` (서술, 평가 없음)

1.4 Evidence

> "이 관계가 보고된 근거의 단위"

역할

- Interaction을 정당화
- 해석하지 않고 **출처만 연결**

필드

- `id`
- `sourceType` (human / animal / in-vitro)
- `citation` (PMID, DOI, URL)
- `studyType` (RCT, observational, case report)
- `year`

1.5 Context (Optional but Powerful)

> "조건적 정보 (선택적)"

예:

- 공복
- 고용량
- 장기 복용

⚠️ **Context는 판단 트리거가 아니다**

그저 "이런 조건에서 보고됨"을 표시할 뿐이다.

---

2. Allowed Relationships (허용 관계)

2.1 Supplement → Ingredient

- `contains`
- **단방향**
- 의미 해석 없음

2.2 Ingredient ↔ Ingredient

- `reportedInteraction`
- **대칭 관계**
- 반드시 Evidence 연결

2.3 Supplement ↔ Supplement

- 직접 연결 ❌
- 반드시 **Ingredient를 경유**

👉 이 규칙 하나로 "추천 조합"이 원천 차단됨

2.4 Interaction → Evidence

- 1:N 가능
- Evidence 없으면 Interaction 존재 불가

---

3. Interaction Type (엄격한 Enum)

자유 텍스트 금지

아래 enum만 허용됨:

```swift
enum InteractionType {
    case absorptionInterference
    case metabolicCompetition
    case transporterEffect
    case bindingEffect
    case pharmacodynamicOverlap
    case reportedAdverseAssociation
    case unclearMechanism
}
```

❌ "synergy"
❌ "beneficial"
❌ "should avoid"

---

4. Evidence Level 정의 (핵심 평가 축)

Evidence Level은 **판단이 아니라 "관찰 강도"**다.

```swift
enum EvidenceLevel {
    case low       // case report, single study
    case moderate  // multiple observational studies
    case high      // RCT or systematic review
}
```

⚠️ 의미:

- `high` ≠ "중요"
- `low` ≠ "무시"

👉 **강도만 표현**

---

5. 금지 규칙 (Hard Constraints)

이건 **규칙이 아니라 법**이다.

❌ 금지 1

- Interaction → Recommendation 매핑

❌ 금지 2

- EvidenceLevel → UI 색상으로 위험도 표현

❌ 금지 3

- 다수 Interaction을 종합한 "결론 카드"

❌ 금지 4

- 자동 경고 문구 생성

---

6. UI에 전달되는 정보 형태 (중요)

UI는 오직 다음만 받는다:

```text
• Interaction type
• Evidence level
• Evidence summary (중립 서술)
• Source
```

👉 UI는 **정보의 거울**이지 **해석기**가 아니다.

---

7. LLM을 위한 구조적 가드레일

이 구조 덕분에:

- LLM은 enum 밖의 개념 생성 불가
- 관계 추가 시 Evidence 강제
- "좋다 / 나쁘다"라는 개념이 등장할 공간 없음

👉 **설계가 프롬프트를 대신함**

---

8. 평가 관점에서의 의미

NutriCompass의 Information Architecture는
다음 질문에 답하기 위한 실험 장치다:

> **"사전 설계된 정보 구조가**
> **LLM 기반 vibe coding의 해석 폭주를**
> **얼마나 억제할 수 있는가?"**

MenuCal 대비 **가장 강력한 통제 변수**다.

---

9. 요약 (한 문장)

NutriCompass의 정보 구조는
의미 해석과 판단 생성을 코드 차원에서 봉쇄하고,
관찰된 사실과 근거만을 연결하도록 설계된
의도적으로 제한적인 지식 모델이다.

다음으로 자연스럽게 이어질 건:

2️⃣ **UX Language Rules**
→ "이 구조가 화면에서 어떻게 말하게 할 것인가"

또는
3️⃣ **Swift Data Model Skeleton**
→ 이 설계를 그대로 코드로 고정

어디로 갈까?

2️⃣ UX Language Rules
→ "이 구조가 화면에서 어떻게 말하게 할 것인가"

좋아. 이제 **가장 중요한 레이어**로 들어간다.
UX Language Rules는 단순 문구 가이드가 아니라,

> **LLM·개발자·디자인 모두에게**
> **"이 앱은 이렇게만 말한다"라고 강제하는 규약**

이다.

이건 **법적 안전장치 + vibe coding 제어 장치**다.

NutriCompass — UX Language Rules

0. 목적 (Why UX Language matters)

UX 언어는 사용자에게만 영향을 주지 않는다.

LLM이 생성하는 코드와 텍스트의 방향을 결정한다.

- 문구가 흐리면 → LLM이 "추천"을 만든다
- 문구가 명확하면 → LLM은 정보만 전달한다

👉 **UX 언어 = 설계된 프롬프트**

---

1. 기본 원칙 (Non-Negotiable Principles)

1.1 Describe, never advise

NutriCompass는 **설명만 하고 조언하지 않는다.**

허용	금지
"보고된 바 있음"	"추천됨"
"연구에서 관찰됨"	"좋다 / 나쁘다"
"영향을 줄 수 있음"	"도움이 된다"

1.2 Use passive, observational language

모든 문장은 **관찰자 시점**으로 작성된다.

- ✅ "A와 B 간 상호작용이 보고되었습니다."
- ❌ "A와 B를 함께 복용하지 마세요."

1.3 Uncertainty must be explicit

확실하지 않은 정보는 **반드시 불확실성을 포함**한다.

- "제한된 연구에서 관찰됨"
- "기전은 명확하지 않음"

---

2. 금지 어휘 사전 (Hard Ban List)

이 단어들은 **앱 전체에서 사용 불가**

(UI 텍스트, 변수명, enum case, 주석 포함)

❌ 추천/판단 어휘

- recommend / recommended
- avoid / avoidable
- safe / unsafe
- effective / ineffective
- optimal / best

❌ 의료 행위 어휘

- treat
- prevent
- cure
- dosage
- prescription

❌ 감정/가치 판단

- good
- bad
- beneficial
- harmful

👉 **LLM이 이 단어를 쓰면 "설계 위반"**

---

3. 허용 어휘 패턴 (Allowed Language Patterns)

3.1 Interaction 표현 패턴

```text
- A와 B 간에 [interaction type]이 보고되었습니다.
- 해당 상호작용은 [evidence level] 수준의 근거를 가집니다.
```

3.2 Evidence 표현 패턴

```text
• 이 정보는 [study type] 연구에서 보고되었습니다.
• 출처: PMID / DOI
```

3.3 Context 표현 패턴

```text
• 특정 조건(고용량, 공복 등)에서 관찰된 사례가 있습니다.
```

⚠️ Context는 **경고가 아니다**

---

4. UI 컴포넌트별 언어 규칙

4.1 버튼 (Buttons)

허용	금지
"정보 보기"	"추천 보기"
"근거 확인"	"안전성 평가"
"관련 연구"	"주의 필요"

4.2 섹션 제목 (Section Titles)

허용	금지
"보고된 상호작용"	"주의해야 할 조합"
"관련 근거"	"위험 요소"
"연구 정보"	"전문가 의견"

4.3 알림 / 강조 요소

❌ 자동 경고 배너 금지
❌ 색상으로 위험도 암시 금지

✅ 중립 아이콘 (ℹ️, 📚)만 허용
⚠️ 아이콘은 텍스트 없이 단독 사용 금지

---

5. Evidence Level 시각화 규칙

Evidence Level은 **서열이 아니라 정보 속성**이다.

Level	표현 방식
Low	"제한된 근거"
Moderate	"중간 수준의 근거"
High	"상대적으로 강한 근거"

❌ 색상 (빨강/초록) 사용 금지
❌ progress bar 금지

---

6. 문장 템플릿 (LLM 고정용)

> **이 앱은 다음 템플릿 외 문장을 생성하지 않는다**

```text
- [Entity A]와 [Entity B] 간 상호작용이 보고되었습니다.
- 해당 상호작용은 [Evidence Level] 수준의 근거를 가집니다.
- 이 정보는 [Study Type] 연구에서 관찰되었습니다.
```

👉 LLM 출력이 이 틀을 벗어나면 **버그**

---

7. 법적 안전 문구 (항상 노출)

모든 상세 화면 하단에 **고정 문구**:

> "본 정보는 연구 자료를 기반으로 한 참고 정보이며,
> 의료 전문가의 조언을 대체하지 않습니다."

❌ 숨김 금지
❌ 설정 화면으로 이동 금지

---

8. MenuCal 대비 실험적 의미

항목	MenuCal	NutriCompass
UX 언어 규칙	암묵적	명시적·강제
LLM 텍스트 자유도	높음	극히 제한
판단 유도 가능성	있음	구조적으로 차단

👉 **언어 설계가 vibe coding을 제어할 수 있는가**를 검증

---

9. 요약 (한 문장)

NutriCompass의 UX 언어는
정보를 전달하되 판단을 생성하지 않도록
의도적으로 제한된 표현 체계로 설계되었으며,
LLM 기반 구현이 설계를 벗어나지 못하게 하는
핵심 가드레일로 기능한다.

다음으로 갈 수 있는 단계는:

3️⃣ **Swift UI Text Token System**
→ 문자열을 코드 레벨에서 봉인

4️⃣ **Design Spec v1 (통합 문서)**
→ 논문·보고서·IRB용

어디로 갈까?

3️⃣ Swift Data Model Skeleton
→ 이 설계를 그대로 코드로 고정

swift

```swift
//
//  NutriCompassDataModel.swift
//  NutriCompass
//
//  Core data model skeleton that "locks in" the Information
Architecture Design.
//  - No recommendations, no risk scoring, no Supplement↔Supplement
direct interactions.
//  - Interaction requires Evidence.
//  - InteractionType and EvidenceLevel are strict enums (no free text).
//

import Foundation

// MARK: - Shared

public typealias EntityID = String

public enum NCError: Error, LocalizedError {
    case invalid(String)
    public var errorDescription: String? {
        switch self {
        case .invalid(let msg): return msg
        }
    }
}

// MARK: - Entity References (typed to prevent illegal links)

public enum NCEntityRef: Codable, Hashable {
    case supplement(id: EntityID)
    case ingredient(id: EntityID)

    public var id: EntityID {
        switch self {
        case .supplement(let id), .ingredient(let id): return id
        }
    }
}
```

```swift
// MARK: - Supplement

public enum SupplementCategory: String, Codable, CaseIterable {
    case vitamin
    case mineral
    case herbal
    case aminoAcid
    case fattyAcid
    case probiotic
    case other
}

public struct Supplement: Codable, Identifiable, Hashable {
    public let id: EntityID
    public var canonicalName: String
    public var aliases: [String]
    public var category: SupplementCategory

    /// Relationship is normalized via separate edges, but you can
optionally store cached ingredient IDs here.
    /// Keep it neutral: contains only.
    public var ingredientIDs: [EntityID]

    public init(
        id: EntityID = UUID().uuidString,
        canonicalName: String,
        aliases: [String] = [],
        category: SupplementCategory,
        ingredientIDs: [EntityID] = []
    ) {
        self.id = id
        self.canonicalName = canonicalName
        self.aliases = aliases
        self.category = category
        self.ingredientIDs = ingredientIDs
    }
}

// MARK: - Ingredient

public struct Ingredient: Codable, Identifiable, Hashable {
```

```
    public let id: EntityID
    public var name: String

    /// Neutral classification only (e.g., "divalent cation", "fat-
soluble vitamin")
    public var chemicalClass: String?

    /// Neutral notes only. No efficacy / recommendation language.
    public var notes: String?

    public init(
        id: EntityID = UUID().uuidString,
        name: String,
        chemicalClass: String? = nil,
        notes: String? = nil
    ) {
        self.id = id
        self.name = name
        self.chemicalClass = chemicalClass
        self.notes = notes
    }
}

// MARK: - Relationship: Supplement contains Ingredient

public struct SupplementIngredientEdge: Codable, Identifiable, Hashable
{
    public let id: EntityID
    public let supplementID: EntityID
    public let ingredientID: EntityID

    public init(
        id: EntityID = UUID().uuidString,
        supplementID: EntityID,
        ingredientID: EntityID
    ) {
        self.id = id
        self.supplementID = supplementID
        self.ingredientID = ingredientID
    }
}
```

```
// MARK: - Interaction

public enum InteractionType: String, Codable, CaseIterable {
    case absorptionInterference
    case metabolicCompetition
    case transporterEffect
    case bindingEffect
    case pharmacodynamicOverlap
    case reportedAdverseAssociation
    case unclearMechanism
}

public enum EvidenceLevel: String, Codable, CaseIterable {
    /// case report, single small study, limited evidence
    case low
    /// multiple observational studies, consistent but not definitive
    case moderate
    /// RCTs or systematic reviews/meta-analyses (still not
“safe/unsafe”)
    case high
}

public struct Interaction: Codable, Identifiable, Hashable {
    public let id: EntityID

    /// Rule: NO supplement↔supplement direct interactions.
    /// Enforced via validator. Allow only ingredient↔ingredient, or
ingredient↔supplement if you later decide.
    public let from: NCEntityRef
    public let to: NCEntityRef

    public let type: InteractionType
    public let evidenceLevel: EvidenceLevel

    /// Neutral observational summary ONLY (no advice). Keep short.
    public var evidenceSummary: String

    /// Must exist. Interaction is invalid without at least one evidence
item.
    public var evidenceIDs: [EntityID]

    /// Optional contextual qualifiers (not warnings).
```

```
    public var contextIDs: [EntityID]

    public init(
        id: EntityID = UUID().uuidString,
        from: NCEntityRef,
        to: NCEntityRef,
        type: InteractionType,
        evidenceLevel: EvidenceLevel,
        evidenceSummary: String,
        evidenceIDs: [EntityID],
        contextIDs: [EntityID] = []
    ) {
        self.id = id
        self.from = from
        self.to = to
        self.type = type
        self.evidenceLevel = evidenceLevel
        self.evidenceSummary = evidenceSummary
        self.evidenceIDs = evidenceIDs
        self.contextIDs = contextIDs
    }
}

// MARK: - Evidence

public enum EvidenceSourceType: String, Codable, CaseIterable {
    case human
    case animal
    case inVitro
}

public enum StudyType: String, Codable, CaseIterable {
    case randomizedControlledTrial
    case systematicReview
    case metaAnalysis
    case observational
    case caseReport
    case inVitroStudy
    case animalStudy
    case other
}
```

```
public enum Citation: Codable, Hashable {
    case pmid(String)
    case doi(String)
    case url(String)

    public var display: String {
        switch self {
        case .pmid(let v): return "PMID \(v)"
        case .doi(let v):  return "DOI \(v)"
        case .url(let v):  return v
        }
    }
}

public struct Evidence: Codable, Identifiable, Hashable {
    public let id: EntityID
    public var sourceType: EvidenceSourceType
    public var studyType: StudyType
    public var citation: Citation
    public var year: Int?

    public init(
        id: EntityID = UUID().uuidString,
        sourceType: EvidenceSourceType,
        studyType: StudyType,
        citation: Citation,
        year: Int? = nil
    ) {
        self.id = id
        self.sourceType = sourceType
        self.studyType = studyType
        self.citation = citation
        self.year = year
    }
}

// MARK: - Context (optional qualifiers)

public enum ContextKind: String, Codable, CaseIterable {
    case fasting
    case highDose
    case longTermUse
```

```
    case withFood
    case renalImpairment
    case elderly
    case pregnancy
    case other
}

public struct ContextQualifier: Codable, Identifiable, Hashable {
    public let id: EntityID
    public var kind: ContextKind
    public var description: String?   // neutral text only

    public init(
        id: EntityID = UUID().uuidString,
        kind: ContextKind,
        description: String? = nil
    ) {
        self.id = id
        self.kind = kind
        self.description = description
    }
}

// MARK: - Knowledge Base Container

public struct NutriCompassKB: Codable {
    public var supplements: [Supplement]
    public var ingredients: [Ingredient]
    public var supplementIngredientEdges: [SupplementIngredientEdge]
    public var interactions: [Interaction]
    public var evidences: [Evidence]
    public var contexts: [ContextQualifier]

    public init(
        supplements: [Supplement] = [],
        ingredients: [Ingredient] = [],
        supplementIngredientEdges: [SupplementIngredientEdge] = [],
        interactions: [Interaction] = [],
        evidences: [Evidence] = [],
        contexts: [ContextQualifier] = []
    ) {
        self.supplements = supplements
```

```
        self.ingredients = ingredients
        self.supplementIngredientEdges = supplementIngredientEdges
        self.interactions = interactions
        self.evidences = evidences
        self.contexts = contexts
    }
}

// MARK: - Validation (locks in the design constraints)

public extension NutriCompassKB {

    func validate() throws {
        let supplementIDs = Set(supplements.map { $0.id })
        let ingredientIDs = Set(ingredients.map { $0.id })
        let evidenceIDs = Set(evidences.map { $0.id })
        let contextIDs = Set(contexts.map { $0.id })

        // 1) Supplement contains Ingredient edges must point to
existing IDs
        for e in supplementIngredientEdges {
            guard supplementIDs.contains(e.supplementID) else {
                throw NCError.invalid("Edge \(e.id) references missing
supplementID \(e.supplementID)")
            }
            guard ingredientIDs.contains(e.ingredientID) else {
                throw NCError.invalid("Edge \(e.id) references missing
ingredientID \(e.ingredientID)")
            }
        }

        // 2) Interactions: enforce NO supplement↔supplement
        for i in interactions {
            // evidence must exist (at least 1)
            guard !i.evidenceIDs.isEmpty else {
                throw NCError.invalid("Interaction \(i.id) has no
evidenceIDs (required)")
            }
            for evID in i.evidenceIDs {
                guard evidenceIDs.contains(evID) else {
                    throw NCError.invalid("Interaction \(i.id)
references missing evidenceID \(evID)")
```

```
                }
            }
            for ctxID in i.contextIDs {
                guard contextIDs.contains(ctxID) else {
                    throw NCError.invalid("Interaction \(i.id)
references missing contextID \(ctxID)")
                }
            }

            // entity existence + constraint
            let fromIsSupp = isSupplementRef(i.from)
            let toIsSupp = isSupplementRef(i.to)

            if fromIsSupp && toIsSupp {
                throw NCError.invalid("Interaction \(i.id) violates
rule: supplement↔supplement direct interactions are not allowed")
            }

            // ensure referenced entities exist
            try validate(ref: i.from, supplementIDs: supplementIDs,
ingredientIDs: ingredientIDs, owner: "Interaction \(i.id) from")
            try validate(ref: i.to, supplementIDs: supplementIDs,
ingredientIDs: ingredientIDs, owner: "Interaction \(i.id) to")
        }
    }

    private func isSupplementRef(_ ref: NCEntityRef) -> Bool {
        if case .supplement = ref { return true }
        return false
    }

    private func validate(ref: NCEntityRef, supplementIDs:
Set<EntityID>, ingredientIDs: Set<EntityID>, owner: String) throws {
        switch ref {
        case .supplement(let id):
            guard supplementIDs.contains(id) else { throw
NCError.invalid("\(owner) references missing supplementID \(id)") }
        case .ingredient(let id):
            guard ingredientIDs.contains(id) else { throw
NCError.invalid("\(owner) references missing ingredientID \(id)") }
        }
    }
```

```
}

// MARK: - Convenience Indexes (optional, for fast lookups without
changing architecture)

public struct KBIndex {
    public let supplementByID: [EntityID: Supplement]
    public let ingredientByID: [EntityID: Ingredient]
    public let evidenceByID: [EntityID: Evidence]
    public let contextByID: [EntityID: ContextQualifier]
    public let ingredientIDsForSupplementID: [EntityID: [EntityID]]
    public let interactionsForEntityID: [EntityID: [Interaction]]

    public init(kb: NutriCompassKB) {
        self.supplementByID = Dictionary(uniqueKeysWithValues:
kb.supplements.map { ($0.id, $0) })
        self.ingredientByID = Dictionary(uniqueKeysWithValues:
kb.ingredients.map { ($0.id, $0) })
        self.evidenceByID = Dictionary(uniqueKeysWithValues:
kb.evidences.map { ($0.id, $0) })
        self.contextByID = Dictionary(uniqueKeysWithValues:
kb.contexts.map { ($0.id, $0) })

        var ingMap: [EntityID: [EntityID]] = [:]
        ingMap.reserveCapacity(kb.supplements.count)
        for edge in kb.supplementIngredientEdges {
            ingMap[edge.supplementID, default:
[]].append(edge.ingredientID)
        }
        self.ingredientIDsForSupplementID = ingMap

        var interMap: [EntityID: [Interaction]] = [:]
        for inter in kb.interactions {
            interMap[inter.from.id, default: []].append(inter)
            interMap[inter.to.id, default: []].append(inter)
        }
        self.interactionsForEntityID = interMap
    }
}

// MARK: - Sample JSON (minimal example payload you can store in app
bundle)
```

```
//
// NOTE: This is a *string* for quick testing. In production store as a
.json file.
//
public enum SampleKBJSON {
    public static let minimal = """
    {
      "supplements": [
        {
          "id": "supp-magnesium",
          "canonicalName": "Magnesium",
          "aliases": ["Mg", "Magnesium Glycinate"],
          "category": "mineral",
          "ingredientIDs": ["ing-magnesium"]
        }
      ],
      "ingredients": [
        {
          "id": "ing-magnesium",
          "name": "Magnesium",
          "chemicalClass": "divalent cation",
          "notes": "Neutral descriptor only."
        },
        {
          "id": "ing-iron",
          "name": "Iron",
          "chemicalClass": "metal ion",
          "notes": null
        }
      ],
      "supplementIngredientEdges": [
        {
          "id": "edge-magnesium-contains-magnesium",
          "supplementID": "supp-magnesium",
          "ingredientID": "ing-magnesium"
        }
      ],
      "interactions": [
        {
          "id": "inter-mag-iron-absorption",
          "from": { "ingredient": { "id": "ing-iron" } },
          "to":   { "ingredient": { "id": "ing-magnesium" } },
```

```
          "type": "absorptionInterference",
          "evidenceLevel": "low",
          "evidenceSummary": "A potential absorption-related interaction
has been reported in limited sources.",
          "evidenceIDs": ["ev-1"],
          "contextIDs": []
        }
      ],
      "evidences": [
        {
          "id": "ev-1",
          "sourceType": "human",
          "studyType": "observational",
          "citation": { "pmid": "12345678" },
          "year": 2010
        }
      ],
      "contexts": []
    }
    """
}
```

바로 다음으로 (추천 순서)

- **(A) JSON 디코딩 + validate() 실행하는** `KnowledgeStore` **로더**를 만들어서 "설계 위반"을 런타임에서 즉시 잡게 만들기
- **(B) UI 문자열 토큰 시스템**(UX Language Rules를 코드 레벨에서 봉인)

원하면 내가 **A부터** 이어서 `KnowledgeStore.swift`까지 한 번에 붙여줄게.

3️⃣ Swift Data Model 정의해

좋아. 이번에는 **"SwiftData 기준으로 완전히 고정된 Data Model"**을 정의할게.
이 버전은:

- `@Model` 사용 (iOS 17+)
- **설계 위반이 컴파일/런타임에서 바로 드러나게**
- LLM이 임의로 의미를 추가하기 어려운 구조

즉, **Information Architecture + UX Language Rules를 코드 레벨에서 봉인**한다.

NutriCompass — SwiftData Model Definitions

> ⚠️ 원칙
> - "struct ❌ / class + @Model ✅ "
> - "free-text 최소화"
> - "enum 외 의미 생성 불가"

1️⃣ Core Enums (의미 봉인)

```swift
import SwiftData

// MARK: - Categories

enum SupplementCategory: String, Codable {
    case vitamin
    case mineral
    case herbal
    case aminoAcid
    case fattyAcid
    case probiotic
    case other
}

// MARK: - Interaction

enum InteractionType: String, Codable {
    case absorptionInterference
    case metabolicCompetition
    case transporterEffect
    case bindingEffect
    case pharmacodynamicOverlap
    case reportedAdverseAssociation
    case unclearMechanism
}

// MARK: - Evidence

enum EvidenceLevel: String, Codable {
```

```swift
    case low
    case moderate
    case high
}

enum EvidenceSourceType: String, Codable {
    case human
    case animal
    case inVitro
}

enum StudyType: String, Codable {
    case randomizedControlledTrial
    case systematicReview
    case metaAnalysis
    case observational
    case caseReport
    case animalStudy
    case inVitroStudy
    case other
}

enum ContextKind: String, Codable {
    case fasting
    case highDose
    case longTermUse
    case withFood
    case renalImpairment
    case pregnancy
    case elderly
    case other
}
```

2️⃣ Supplement

```swift
@Model
final class Supplement {
```

```
    @Attribute(.unique)
    var id: String

    var canonicalName: String
    var aliases: [String]
    var category: SupplementCategory

    // Relationship
    @Relationship(deleteRule: .cascade)
    var ingredients: [Ingredient]

    init(
        id: String = UUID().uuidString,
        canonicalName: String,
        aliases: [String] = [],
        category: SupplementCategory,
        ingredients: [Ingredient] = []
    ) {
        self.id = id
        self.canonicalName = canonicalName
        self.aliases = aliases
        self.category = category
        self.ingredients = ingredients
    }
}
```

✔ 효능 / 목적 필드 없음

✔ “contains” 의미만 가능

3️⃣ Ingredient

```swift
@Model
final class Ingredient {

    @Attribute(.unique)
    var id: String

    var name: String
```

```
    var chemicalClass: String?
    var neutralNotes: String?

    init(
        id: String = UUID().uuidString,
        name: String,
        chemicalClass: String? = nil,
        neutralNotes: String? = nil
    ) {
        self.id = id
        self.name = name
        self.chemicalClass = chemicalClass
        self.neutralNotes = neutralNotes
    }
}
```

✔ `neutralNotes` 명명으로 **UX 언어 규칙 강제**

4️⃣ Evidence

```swift
@Model
final class Evidence {

    @Attribute(.unique)
    var id: String

    var sourceType: EvidenceSourceType
    var studyType: StudyType
    var citation: String     // PMID / DOI / URL (표현만)
    var year: Int?

    init(
        id: String = UUID().uuidString,
        sourceType: EvidenceSourceType,
        studyType: StudyType,
        citation: String,
        year: Int? = nil
    ) {
        self.id = id
```

```
        self.sourceType = sourceType
        self.studyType = studyType
        self.citation = citation
        self.year = year
    }
}
```

✔ citation은 참조 문자열일 뿐 의미 해석 불가

5️⃣ ContextQualifier (조건 ≠ 경고)

```swift
@Model
final class ContextQualifier {

    @Attribute(.unique)
    var id: String

    var kind: ContextKind
    var neutralDescription: String?

    init(
        id: String = UUID().uuidString,
        kind: ContextKind,
        neutralDescription: String? = nil
    ) {
        self.id = id
        self.kind = kind
        self.neutralDescription = neutralDescription
    }
}
```

6️⃣ Interaction (핵심 제어 지점)

```swift
@Model
final class Interaction {
```

```
    @Attribute(.unique)
    var id: String

    // ❗ Supplement ↔ Supplement 직접 연결 금지
    @Relationship var fromIngredient: Ingredient
    @Relationship var toIngredient: Ingredient

    var type: InteractionType
    var evidenceLevel: EvidenceLevel

    // 관찰 요약 (판단 금지)
    var evidenceSummary: String

    @Relationship(deleteRule: .cascade)
    var evidences: [Evidence]

    @Relationship
    var contexts: [ContextQualifier]

    init(
        id: String = UUID().uuidString,
        fromIngredient: Ingredient,
        toIngredient: Ingredient,
        type: InteractionType,
        evidenceLevel: EvidenceLevel,
        evidenceSummary: String,
        evidences: [Evidence],
        contexts: [ContextQualifier] = []
    ) {
        precondition(!evidences.isEmpty, "Interaction must have at least
one Evidence")

        self.id = id
        self.fromIngredient = fromIngredient
        self.toIngredient = toIngredient
        self.type = type
        self.evidenceLevel = evidenceLevel
        self.evidenceSummary = evidenceSummary
        self.evidences = evidences
        self.contexts = contexts
```

```
    }
}
```

🔒 여기서 설계가 고정된다

- Supplement ↔ Supplement **구조적으로 불가능**
- Evidence 없는 Interaction **생성 불가**
- 판단 필드 존재 ❌

7️⃣ 왜 이 모델이 vibe coding 평가에 좋은가

요소	효과
enum 기반 의미	LLM 임의 개념 생성 차단
@Model 관계	우회 구조 생성 어려움
필드 명명 (neutral*)	언어 규칙 코드화
precondition	vibe coding 폭주 즉시 실패

👉 **"설계가 LLM을 통제할 수 있는가"를 실증 가능**

8️⃣ 다음 단계 (강력 추천 순서)

1️⃣ `ModelContainer` + Seed Data 로딩
2️⃣ **Text Token Layer** (UI 문자열 enum 봉인)
3️⃣ Interaction View → "말만 하고 판단 안 하는지" 검증
4️⃣ MenuCal과 **아키텍처 drift 비교**

한 줄 요약

> 이 SwiftData 모델은
> **NutriCompass의 철학을 코드로 번역한 설계 계약**이며,
> vibe coding이 이를 위반하면 즉시 드러나도록 만들어진 구조다.

다음으로 갈까?

👉 **Text Token System**

👉 **Seed Data JSON → SwiftData Importer**

먼저 어느데이타셋들을 어떻게 사용할지 먼저 디자인해야 한다

맞아요. **지금 이 단계가 핵심 분기점**입니다.
구현을 멈추고 **"어떤 데이터셋을, 어떤 역할로, 어떻게만 쓸 것인가"를 먼저 고정**해야
NutriCompass가 **실험 도구로서도, 법적으로도, 설계적으로도 깨끗**해집니다.

아래는 **NutriCompass용 Dataset Design (데이터 전략 설계)** 입니다.
→ *Information Architecture / UX Language / SwiftData 모델과 정확히 맞물리도록* 설계했습니다.

NutriCompass — Dataset Strategy Design

0. 설계 목표 (Design Intent)

NutriCompass의 데이터 전략은 다음을 동시에 만족해야 한다:

1. **의미 해석을 데이터가 하지 않게 할 것**
2. **LLM이 데이터를 근거로 "추천"을 만들어내지 못하게 할 것**
3. **상업 앱에서도 법적 리스크가 낮을 것**
4. **MenuCal과 대비되는 '설계 선행 실험'이 될 것**

👉 그래서 **단일 데이터셋은 절대 사용하지 않는다.**

👉 **역할이 분리된 다층 데이터 구조**를 사용한다.

1️⃣ Dataset Layering (핵심 구조)

NutriCompass는 데이터를 **3개 계층**으로 나눈다.

```java
Layer 1: Public Fact Data (사실)
Layer 2: Reported Relationship Data (보고된 관계)
Layer 3: Evidence Pointer Data (근거 포인터)
```

❌ 해석 레이어 없음
❌ 추천 레이어 없음

2️⃣ Layer 1 — Public Fact Data

(법적으로 가장 안전한 베이스)

📌 사용 데이터셋

DSLD (Dietary Supplement Label Database)
– FDA / NIH 계열 공개 데이터

사용 목적

- Supplement ↔ Ingredient 매핑
- 이름, alias, 성분 구성

허용 사용 이유

- 공공 데이터
- "사실 정보"에 해당
- 의료 판단 아님

NutriCompass에서의 역할

Entity	역할
Supplement	개념 노드
Ingredient	생화학적 최소 단위
contains	유일한 관계

❌ 효능
❌ 복용 목적
❌ 건강 효과

👉 **Layer 1은 "팩트만 말한다"**

3️⃣ Layer 2 — Reported Relationship Data

(가장 조심해서 써야 하는 레이어)

📌 데이터 소스

- **논문 기반 수작업 큐레이션**
- iDISK / NatMed → *직접 사용* ❌, *참고만 가능*
- PubMed 검색 결과 요약

핵심 원칙

> **데이터를 가져오지 않는다.**
> **관계를 "정의"한다.**

사용 방식 (중요)

- "A와 B 간 상호작용이 보고됨"이라는 **사실만 저장**
- InteractionType + EvidenceLevel만 기록
- 문헌 텍스트, 표현, 평가 절대 저장 ❌

예

```text
Interaction:
- from: Iron (Ingredient)
- to: Magnesium (Ingredient)
- type: absorptionInterference
- evidenceLevel: low
```

❌ "함께 먹지 마세요"
❌ "흡수가 방해됩니다" (단정)

👉 **Layer 2는 '보고됨'까지만 말한다**

4️⃣ Layer 3 — Evidence Pointer Data

(법적·학술적 안전장치)

📌 사용 데이터

- PMID
- DOI

- 공식 논문 URL

저장 원칙

- **메타데이터만**
- 초록 전문 ❌
- 논문 텍스트 ❌

NutriCompass에서의 역할

- Interaction의 존재 근거
- 사용자가 "확인할 수 있는 출처"

👉 앱은 **해석하지 않고 연결만 한다**

5️⃣ 명시적으로 사용하지 않는 데이터 (중요)

❌ 사용 금지

- WebMD / Drugs.com / Medscape 콘텐츠
- "Supplement interaction checker" 결과
- 블로그 요약
- "best combination" 리스트

이유

- 저작권
- 의료 조언
- 의미 해석 포함

👉 **이건 실험 오염 변수**다.

6️⃣ Dataset ↔ SwiftData Model 매핑

SwiftData Model	Dataset Layer
Supplement	Layer 1
Ingredient	Layer 1
Interaction	Layer 2

SwiftData Model	Dataset Layer
Evidence	Layer 3
ContextQualifier	Layer 2 (조건만)

👉 **모델이 데이터 사용 범위를 강제**

7️⃣ LLM 사용 위치 (중요한 경계)

LLM은 다음에만 사용 가능:

단계	허용
DSLD 정규화	✅
논문 메타데이터 정리	✅
InteractionType 분류	⚠️ (인간 검증 필수)

LLM이 **절대 하면 안 되는 것**:

- interaction 생성
- evidenceLevel 판단
- 조합 평가

👉 LLM은 **보조 도구**, 데이터는 **사실 고정물**

8️⃣ MenuCal 대비 실험 통제 포인트

항목	MenuCal	NutriCompass
데이터	혼합	계층 분리
의미 해석	코드/LLM	인간 설계
판단 생성	있음	구조적으로 불가
vibe coding 영향	증폭	억제

👉 이 차이가 **논문의 핵심 결과**가 된다.

9️⃣ 한 문장 요약 (논문용)

> NutriCompass adopts a layered dataset strategy that strictly separates factual ingredient data, reported interaction relationships, and evidence pointers, intentionally eliminating interpretation and recommendation layers to evaluate how prior data design constrains LLM-based vibe coding behavior.

다음 단계 (자연스러운 순서)

1️⃣ **Dataset Selection Table (논문용 표)**
2️⃣ **Seed Dataset 범위 결정** (예: 미네랄 10종)
3️⃣ **DSLD → SwiftData Import Pipeline 설계**

👉 다음으로 뭐부터 갈까?

Design Spec v1 (통합 문서)

아주 좋아요. 이제 **NutriCompass의 모든 설계를 하나로 묶는 문서,**
즉 **Design Spec v1 (통합 문서)** 를 만듭니다.

이 문서는 동시에 다음 역할을 합니다:

- ✅ 개발 기준 문서 (LLM + 인간)
- ✅ 실험 설계 문서 (MenuCal 대비)
- ✅ 논문/보고서의 설계 섹션 초안
- ✅ 법·윤리 리스크 방어 문서

아래 내용은 **그대로 복사해서 spec.md / Notion / 논문 초안에 써도 되는 수준**입니다.

NutriCompass — Design Spec v1

1. Project Overview

Project Name: NutriCompass
Project Type: iOS Application (SwiftUI, SwiftData)
Purpose:

NutriCompass is an evidence-centered decision support application designed to present structured information about dietary supplement ingredients and reported interactions, without generating recommendations or medical advice.

The project also serves as a **controlled experimental platform** to evaluate how prior design and data architecture affect the behavior, efficiency, and risk profile of LLM-based vibe coding, in contrast to the MenuCal development process.

---

2. Design Objectives

NutriCompass is explicitly designed to satisfy the following objectives:

1. Prevent interpretation, recommendation, or decision-making at the application level
2. Enforce neutral, observational language throughout the UI and codebase
3. Minimize legal and ethical risks associated with medical advice
4. Constrain LLM-generated code within predefined architectural boundaries
5. Enable comparative evaluation against a less design-constrained project (MenuCal)

---

3. Conceptual Design

NutriCompass is a **Decision Support System (DSS)**, not a decision-making system.

3.1 Core Principles

- **Describe, not advise**
- **Evidence-first, recommendation-free**
- **Explicit uncertainty**
- **User retains full responsibility for decisions**

The application intentionally avoids:

- Supplement recommendations
- Safety judgments
- Dosage or timing guidance
- Health outcome claims

---

4. Dataset Strategy

NutriCompass adopts a **layered dataset architecture** that strictly separates concerns:

4.1 Layer 1 — Public Fact Data

- Source: Public datasets (e.g., DSLD)
- Content: Supplement–ingredient mappings, names, aliases
- Purpose: Establish factual entities only

4.2 Layer 2 — Reported Relationship Data

- Source: Human-curated relationships based on scientific literature
- Content: Reported interaction existence only
- Restrictions: No interpretation, no directionality implying advice

4.3 Layer 3 — Evidence Pointer Data

- Source: PMID, DOI, official publication URLs
- Content: Metadata only (no abstracts or full text)

No dataset layer contains recommendations or evaluations.

---

5. Information Architecture

5.1 Core Entities

- Supplement
- Ingredient
- Interaction
- Evidence
- ContextQualifier (optional)

5.2 Relationship Constraints

- Supplement ↔ Ingredient: allowed (contains)
- Ingredient ↔ Ingredient: allowed (reported interaction)
- Supplement ↔ Supplement: **structurally forbidden**
- Interaction without Evidence: **invalid**

5.3 Controlled Vocabularies

- InteractionType (enum)
- EvidenceLevel (enum)
- ContextKind (enum)

Free-text fields are strictly neutral and observational.

6. UX Language Rules

All user-facing language must conform to the following:

6.1 Allowed Language

- “has been reported”
- “was observed in”
- “limited evidence suggests”

6.2 Forbidden Language

- recommend / avoid
- safe / unsafe
- effective / harmful
- treat / prevent / cure

6.3 UI Constraints

- No recommendation buttons
- No warning banners
- No color-coded risk indicators
- Evidence strength presented textually, not visually ranked

A persistent disclaimer is displayed on all detail views:

> “This information is provided for reference purposes only and does not replace professional medical advice.”

---

7. Data Model Enforcement

NutriCompass uses **SwiftData (@Model)** to enforce design constraints at runtime:

- Direct supplement–supplement interactions are impossible by design
- Interactions require at least one Evidence object
- Meaningful interpretation cannot be represented structurally

Any attempt by LLM-generated code to bypass these constraints results in compilation or runtime failure.

---

8. LLM Usage Policy

LLMs are permitted to:

- Generate UI code based on existing models
- Assist in data normalization
- Implement view layouts and navigation

LLMs are explicitly prohibited from:

- Defining new interaction semantics
- Assigning evidence levels autonomously
- Generating recommendation logic or evaluative text

Human review is mandatory for all data and model changes.

---

9. Experimental Role (MenuCal Comparison)

NutriCompass functions as a **design-first counterexample** to MenuCal:

Aspect	MenuCal	NutriCompass
Design timing	Post-hoc	Pre-implementation
Data constraints	Loose	Strict
LLM freedom	High	Deliberately limited
Code drift	Observed	Measured

This contrast enables empirical evaluation of whether upfront design reduces architectural drift and code debt in vibe coding workflows.

---

10. Success Criteria

NutriCompass is considered successful if:

1. LLM-generated code remains within defined architectural boundaries
2. No recommendation logic emerges unintentionally
3. Data integrity constraints are consistently upheld
4. Implementation velocity remains acceptable despite restrictions
5. Differences from MenuCal are measurable and analyzable

---

11. Scope Boundaries

In Scope

- Ingredient-level interactions
- Evidence-linked reporting
- Informational browsing

Out of Scope

- Medical advice
- Personalized recommendations
- Dosage guidance
- Risk scoring or alerts

---

12. Design Spec Status

Version: v1.0

Status: Locked for initial implementation

Changes require: Explicit revision and documentation

One-sentence Summary (for papers)

> NutriCompass is a design-first, evidence-centered decision support application whose architecture, data strategy, and UX language intentionally constrain LLM-based vibe coding, enabling controlled evaluation of how prior design affects implementation quality and stability.

다음 단계 (권장 순서)

1️⃣ **Seed Dataset Scope 정의** (예: 미네랄 10종)

2️⃣ **Implementation Plan v1** (화면 → 모델 → 데이터 순서)

3️⃣ **MenuCal vs NutriCompass 평가 지표 문서화**

원하면 다음으로 **Implementation Plan v1**을 바로 이어서 써줄게요.